

Campus Message Detective — Naive Bayes Classifier

Important or Promotional?

Imagine a college student receives **50–100 messages every week**:

- academic announcements
- assignment and exam reminders
- placement notifications
- event information
- discounts, prizes and promotional messages

The challenge:

Can a machine learn from previously labelled messages and automatically decide whether a new message is Important or Promotional?

In this mini-project, we will build a **Naive Bayes text classifier**.

Suppose the college has an **AI-powered message filter**.

A new message arrives:

"Submit your project report before 5 PM tomorrow."

The model asks:

Which class is more probable?

- $P(\text{Important} \mid \text{message})$
- $P(\text{Promotional} \mid \text{message})$

Naive Bayes uses the evidence contained in the words of the message to compare these probabilities.

The word **"submit"**, for example, may occur frequently in Important messages, while words such as **"free"**, **"offer"**, **"win"**, and **"claim"** may be strongly associated with Promotional messages.

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

print("Libraries imported successfully!")
```

Libraries imported successfully!

✓ 1. Create our mini dataset

To keep the project self-contained, we will create a small dataset directly in the notebook.

Each record contains:

- `message` → the text received by the student
- `label` → the correct class

This is a **teaching dataset**, not a production spam dataset.

```
# Create the dataset
data = [('Buy one get one free. Shop now and save big.', 'Promotional'), ('T

df = pd.DataFrame(data, columns=["message", "label"])

print("Dataset shape:", df.shape)
display(df.head(10))
```

Dataset shape: (56, 2)

	message	label	
0	Buy one get one free. Shop now and save big.	Promotional	
1	The attendance report will be finalized this w...	Important	
2	Reminder: the coding contest closes tonight, r...	Important	
3	The classroom for today's lecture has changed ...	Important	
4	The placement aptitude test is scheduled for S...	Important	
5	Students selected for the internship should re...	Important	
6	The student portal is under maintenance from 8...	Important	
7	Please confirm your participation in the works...	Important	
8	Mega sale starts now. Up to 70 percent discount.	Promotional	
9	The coding contest registration closes tonight.	Important	

```
# Inspect the dataset
print("Class distribution:")
display(df["label"].value_counts())

print("\nMissing values:")
print(df.isnull().sum())

# Visualize class distribution
df["label"].value_counts().plot(kind="bar", rot=0, figsize=(6,4))
plt.title("Number of Messages in Each Class")
plt.xlabel("Class")
plt.ylabel("Number of Messages")
plt.show()
```

Class distribution:

	count
label	
Important	29
Promotional	27

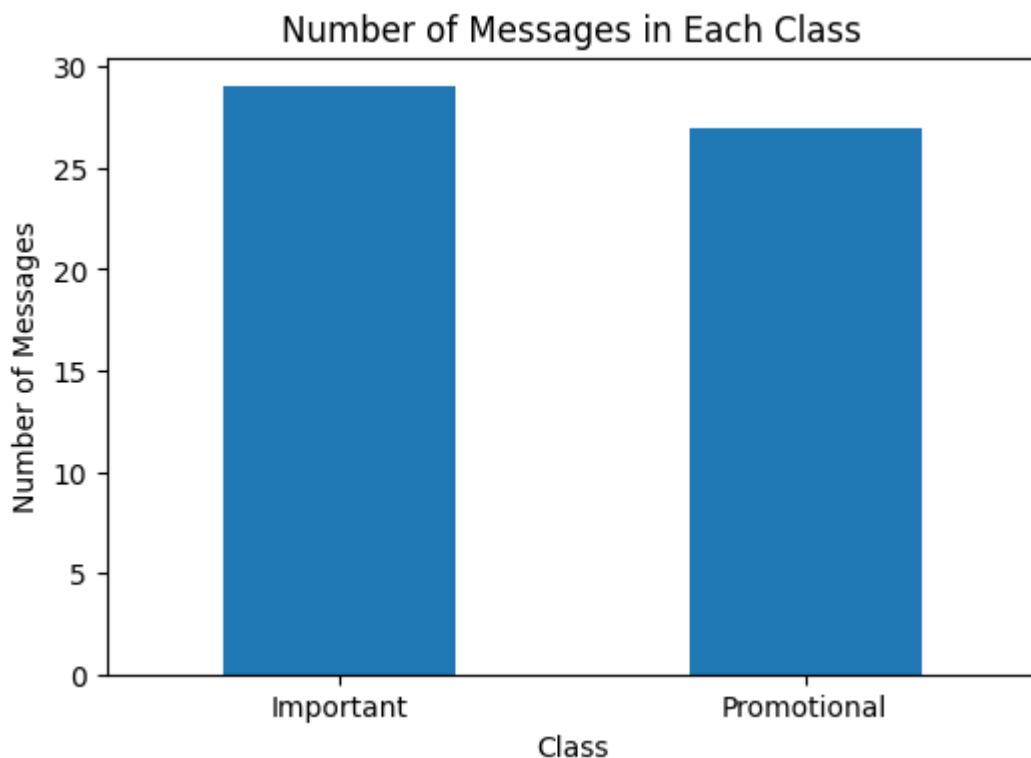
dtype: int64

Missing values:

message 0

label 0

dtype: int64



✓ 2. Why do we need to convert text into numbers?

A machine-learning algorithm cannot directly work with:

"Submit your assignment before Friday"

It needs numerical features.

One simple approach is **Bag of Words (BoW)**.

Suppose our tiny dataset contains:

- "free gift today"
- "submit assignment today"

The vocabulary might become:

[assignment, free, gift, submit, today]

A message is then represented by word counts.

For example:

"free gift today" → [0, 1, 1, 0, 1]

This is exactly the type of representation that **Multinomial Naive Bayes** works very well with.

```
# Separate input (X) and target (y)
X = df["message"]
y = df["label"]

# Split before fitting the vectorizer to avoid data leakage
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("Training messages:", len(X_train))
print("Testing messages :", len(X_test))
```

```
Training messages: 42
Testing messages : 14
```

```
# Convert text into Bag-of-Words features
vectorizer = CountVectorizer(
    lowercase=True,
    stop_words="english"
)

X_train_bow = vectorizer.fit_transform(X_train)
X_test_bow = vectorizer.transform(X_test)

print("Training matrix shape:", X_train_bow.shape)
print("Testing matrix shape :", X_test_bow.shape)

print("\nFirst 30 vocabulary words:")
print(vectorizer.get_feature_names_out()[:30])
```

```
Training matrix shape: (42, 157)
Testing matrix shape : (14, 157)
```

```
First 30 vocabulary words:
['10' '11' '204' '30' '70' 'achievement' 'activate' 'ai' 'aptitude'
 'assessment' 'assignment' 'attendance' 'available' 'bank' 'big' 'biggest'
 'bonus' 'brand' 'bring' 'buy' 'campus' 'card' 'cashback' 'certificate'
 'chance' 'changed' 'check' 'chosen' 'claim' 'class']
```

```
# See what Bag-of-Words actually looks like
bow_sample = pd.DataFrame(
    X_train_bow[:5].toarray(),
    columns=vectorizer.get_feature_names_out()
)

display(bow_sample)
```

	10	11	204	30	70	achievement	activate	ai	aptitude	assessment	...
0	0	0	0	0	0	0	0	0	0	0	...
1	0	0	0	0	0	0	0	0	0	0	...
2	0	0	0	0	0	0	0	0	0	0	...
3	0	0	0	0	0	0	0	0	0	0	...
4	0	0	0	0	0	0	0	0	0	0	...

5 rows × 157 columns

3. The Naive Bayes idea

Bayes' theorem tells us how to update a probability after observing evidence.

For a class C and observed message X :

Posterior $\propto$ **Likelihood** $\times$ **Prior**

For text classification, Naive Bayes estimates:

- **Prior:** How common is each class?
- **Likelihood:** How frequently do words occur in each class?
- **Posterior:** Given these words, which class is most likely?

Why is it called "Naive"?

Because it makes a simplifying assumption:

It treats the features (words) as conditionally independent given the class.

Natural language does **not** really satisfy this assumption.

Nevertheless, this simple assumption often works surprisingly well for text classification.

The multinomial version is especially suitable when features represent **word counts**.

4. Train the Multinomial Naive Bayes model

The workflow is now very small:

Text → Bag of Words → Naive Bayes → Prediction

We use `MultinomialNB` because our features are word counts.

```
# Create and train the Naive Bayes classifier
model = MultinomialNB()

model.fit(X_train_bow, y_train)

print("Naive Bayes model trained successfully!")
```

Naive Bayes model trained successfully!

```
# Make predictions
y_pred = model.predict(X_test_bow)

print("Predictions:")
comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred,
    "Message": X_test.values
})

display(comparison)
```

Predictions:

	Actual	Predicted	Message
0	Important	Important	Please confirm your participation in the works...
1	Important	Promotional	The timetable for the next semester is now ava...
2	Promotional	Promotional	Get 80 percent off on shoes. Limited time offer!
3	Promotional	Promotional	Act now to receive your complimentary gift.
4	Important	Important	The NLP practical will be conducted in Lab 3 t...
5	Important	Important	The university exam timetable has been published.
6	Important	Important	The department meeting for student representat...
7	Promotional	Important	Congratulations winner! Send your details to r...
8	Promotional	Promotional	Unlock your exclusive coupon and save money to...
9	Promotional	Promotional	Get instant cashback on your next purchase.
10	Promotional	Promotional	Earn easy money from home. Join our program to...
11	Promotional	Promotional	Congratulations! You won a free smartphone. Cl...
12	Important	Important	The project review is scheduled for Monday mor...
13	Important	Important	The coding contest registration closes tonight.



Next steps:

[Generate code with comparison](#)[New interactive sheet](#)

✓ 5. Evaluate the classifier

Accuracy tells us the overall percentage of correct predictions.

But we also want to know:

- Which Important messages were correctly identified?
- Which Promotional messages were correctly identified?
- Did the model confuse one class with the other?

For this, we use a **confusion matrix** and **classification report**.

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2%}")

# Detailed metrics
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

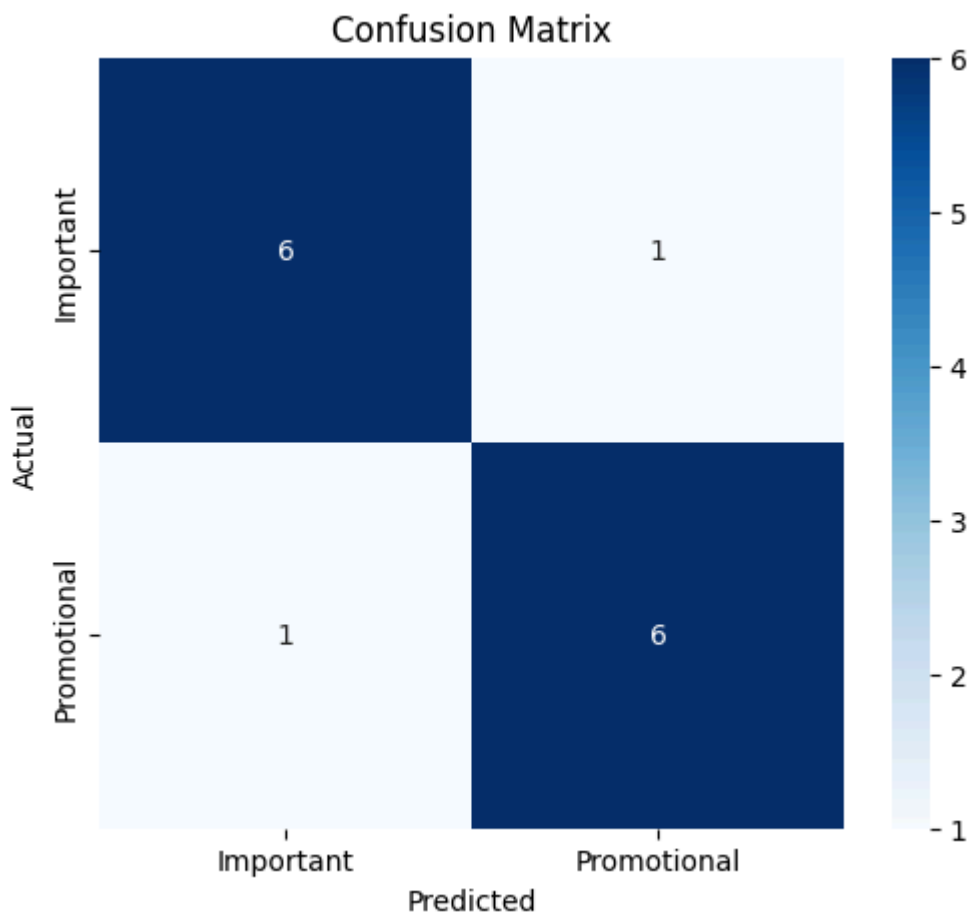
# Confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=["Important", "Promotional"])
```

```
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Important", "Promotional"],
    yticklabels=["Important", "Promotional"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Accuracy: 85.71%

Classification Report:

	precision	recall	f1-score	support
Important	0.86	0.86	0.86	7
Promotional	0.86	0.86	0.86	7
accuracy			0.86	14
macro avg	0.86	0.86	0.86	14
weighted avg	0.86	0.86	0.86	14



6. classify brand-new messages

Our model has never seen these messages before.

Let's become the student and test it!

```
new_messages = [
    "Please submit your machine learning assignment by tomorrow.",
    "Congratulations! Claim your free gift now.",
    "The placement aptitude test starts at 9 AM on Saturday.",
    "Win a brand new phone by clicking this link.",
    "The seminar hall has been changed to Room 301.",
    "Get 70 percent discount today. Limited offer!",
    "Your project viva is scheduled for Monday.",
    "You have won an exclusive cashback reward."
]

new_bow = vectorizer.transform(new_messages)
new_predictions = model.predict(new_bow)
new_probabilities = model.predict_proba(new_bow)

results = pd.DataFrame({
    "Message": new_messages,
    "Prediction": new_predictions,
    "Confidence": new_probabilities.max(axis=1)
})

display(results)
```

	Message	Prediction	Confidence	
0	Please submit your machine learning assignment...	Important	0.992164	
1	Congratulations! Claim your free gift now.	Promotional	0.981202	
2	The placement aptitude test starts at 9 AM on ...	Important	0.933639	
3	Win a brand new phone by clicking this link.	Promotional	0.986447	
4	The seminar hall has been changed to Room 301.	Important	0.956654	
5	Get 70 percent discount today. Limited offer!	Promotional	0.995640	
6	Your project viva is scheduled for Monday.	Important	0.920259	
7	You have won an exclusive cashback reward.	Promotional	0.981202	

Next steps:

[Generate code with results](#)
[New interactive sheet](#)

7. Look inside the model

One of the best ways to understand Naive Bayes is to inspect the words associated with each class.

The model stores estimated log-probabilities for every vocabulary word.

A useful classroom question:

Which words have the strongest association with Promotional messages?

Let's find out.

```
# Inspect important words for each class
feature_names = vectorizer.get_feature_names_out()

class_names = model.classes_

for class_index, class_name in enumerate(class_names):
    log_probs = model.feature_log_prob_[class_index]
    top_indices = np.argsort(log_probs)[-10:][::-1]

    print(f"\nTop words associated with: {class_name}")
    for idx in top_indices:
        print(f"{feature_names[idx]:15s} {log_probs[idx]:.3f}")
```

Top words associated with: Important

pm	-3.821
students	-4.227
tomorrow	-4.227
lecture	-4.227
submit	-4.515
report	-4.515
office	-4.515
practical	-4.515
portal	-4.515
seminar	-4.515

Top words associated with: Promotional

free	-3.777
special	-3.959
win	-4.182
reward	-4.182
offer	-4.182
sale	-4.182
discount	-4.182
claim	-4.182
click	-4.182
shop	-4.470

Test the Naive Bayes Model with a New Message

```

# Take message from the user
new_message = input("Enter your message: ")

# Convert the message into the same Bag-of-Words representation
new_message_vector = vectorizer.transform([new_message])

# Make prediction
prediction = model.predict(new_message_vector)[0]

# Get prediction probabilities
probabilities = model.predict_proba(new_message_vector)[0]

# Get class names
classes = model.classes_

# Display result
print("\n" + "=" * 50)
print("                PREDICTION RESULT")
print("=" * 50)

print(f"\nMessage: {new_message}")

if prediction == "Important":
    print("\n Prediction: IMPORTANT")
else:
    print("\n Prediction: PROMOTIONAL")

# Display probabilities
print("\nProbability:")
for class_name, probability in zip(classes, probabilities):
    print(f"    {class_name}: {probability:.2%}")

print(f"\nModel Confidence: {max(probabilities):.2%}")
print("=" * 50)

```

Enter your message: you will get a free gift

```

=====
                PREDICTION RESULT
=====

```

Message: you will get a free gift

Prediction: PROMOTIONAL

Probability:

Important: 7.73%

Promotional: 92.27%

Model Confidence: 92.27%

```

=====

```

```

import joblib

```

```

# Save trained model

```

```
joblib.dump(model, "naive_bayes_model.pkl")

# Save text vectorizer
joblib.dump(vectorizer, "vectorizer.pkl")

print("Model and vectorizer saved successfully!")

Model and vectorizer saved successfully!
```

✓ HTML Code:

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Campus Message Detective</title>

    <style>

        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }

        body {
            font-family: Arial, sans-serif;
            background: linear-gradient(135deg, #eef2ff, #f8fafc, #e0f2fe);
            min-height: 100vh;
            padding: 40px 20px;
            color: #1e293b;
        }

        .container {
            max-width: 950px;
            margin: auto;
        }

        .header {
            text-align: center;
            margin-bottom: 30px;
        }

        .icon {
            font-size: 55px;
            margin-bottom: 10px;
        }
```

```
.header h1 {
  font-size: 36px;
  color: #1e3a8a;
  margin-bottom: 10px;
}

.header p {
  color: #64748b;
  font-size: 17px;
}

.card {
  background: white;
  border-radius: 20px;
  padding: 30px;
  margin-bottom: 25px;
  box-shadow: 0 10px 30px rgba(15, 23, 42, 0.10);
}

.card h2 {
  margin-bottom: 10px;
}

.description {
  color: #64748b;
  margin-bottom: 25px;
  line-height: 1.6;
}

label {
  display: block;
  font-weight: bold;
  margin-bottom: 8px;
}

select,
textarea {
  width: 100%;
  padding: 14px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  font-size: 15px;
  background: #f8fafc;
  outline: none;
}

select:focus,
textarea:focus {
  border-color: #2563eb;
  background: white;
}

textarea {
  min-height: 130px;
}
```

```
        resize: vertical;
        margin-bottom: 20px;
    }

    .or {
        text-align: center;
        margin: 20px 0;
        color: #94a3b8;
        font-weight: bold;
    }

    .button-container {
        text-align: center;
        margin-top: 20px;
    }

    button {
        border: none;
        padding: 14px 35px;
        border-radius: 10px;
        background: #2563eb;
        color: white;
        font-size: 16px;
        font-weight: bold;
        cursor: pointer;
    }

    button:hover {
        background: #1d4ed8;
    }

    .result {
        text-align: center;
    }

    .prediction {
        display: inline-block;
        padding: 12px 25px;
        border-radius: 30px;
        font-size: 20px;
        font-weight: bold;
        margin-bottom: 20px;
    }

    .important {
        background: #dcfce7;
        color: #166534;
    }

    .promotional {
        background: #fef3c7;
        color: #92400e;
    }

    .category {
```

```
    display: inline-block;
    background: #eff6ff;
    color: #1d4ed8;
    padding: 6px 12px;
    border-radius: 20px;
    font-size: 13px;
    font-weight: bold;
    margin-bottom: 15px;
}

.message-box {
    background: #f8fafc;
    padding: 18px;
    border-radius: 10px;
    text-align: left;
    margin-top: 15px;
    line-height: 1.6;
}

.probabilities {
    margin-top: 25px;
    text-align: left;
}

.probabilities h3 {
    margin-bottom: 20px;
}

.probability-row {
    margin-bottom: 20px;
}

.probability-header {
    display: flex;
    justify-content: space-between;
    margin-bottom: 7px;
    font-weight: bold;
}

.bar {
    width: 100%;
    height: 12px;
    background: #e2e8f0;
    border-radius: 20px;
    overflow: hidden;
}

.bar-fill {
    height: 100%;
    background: #2563eb;
    border-radius: 20px;
}

.message-list {
    display: grid;
```

```
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 15px;
    margin-top: 20px;
  }

  .message-item {
    padding: 18px;
    border: 1px solid #e2e8f0;
    border-radius: 12px;
    background: #f8fafc;
  }

  .message-item h3 {
    color: #1e3a8a;
    font-size: 16px;
    margin-bottom: 8px;
  }

  .message-item p {
    color: #64748b;
    font-size: 14px;
    line-height: 1.5;
  }

  .footer {
    text-align: center;
    color: #94a3b8;
    margin-top: 30px;
    font-size: 14px;
  }

  @media (max-width: 600px) {

    body {
      padding: 20px 12px;
    }

    .card {
      padding: 20px;
    }

    .header h1 {
      font-size: 28px;
    }

    button {
      width: 100%;
    }

  }

</style>

</head>
```

```

<body>

<div class="container">

  <!-- HEADER -->

  <div class="header">

    <div class="icon">🎓</div>

    <h1>Campus Message Detective</h1>

    <p>
      Naive Bayes Classifier for College Messages
    </p>

  </div>

  <!-- CLASSIFIER -->

  <div class="card">

    <h2>📩 Classify a Message</h2>

    <p class="description">

      Select a college message or enter your own message.
      Our trained Naive Bayes model will classify it as
      <strong>Important</strong> or
      <strong>Promotional</strong>.

    </p>

    <form method="POST">

      <!-- SAMPLE MESSAGE -->

      <label for="selected_message">
        Select a College Message
      </label>

      <select
        name="selected_message"
        id="selected_message"
      >

        <option value="">
          -- Choose a message --
        </option>

```

```

        {% for item in messages %}

        <option value="{{ loop.index0 }}">

            {{ item.category }} -
            {{ item.message }}

        </option>

        {% endfor %}

    </select>

    <!-- OR -->

    <div class="or">
        OR
    </div>

    <!-- CUSTOM MESSAGE -->

    <label for="custom_message">
        Enter Your Own Message
    </label>

    <textarea
        name="custom_message"
        id="custom_message"
        placeholder="Example: Submit your DBMS assignment before tom
    ></textarea>

    <!-- BUTTON -->

    <div class="button-container">

        <button type="submit">
             Classify Message
        </button>

    </div>

</form>

</div>

<!-- RESULT -->

{% if prediction %}

<div class="card result">

```

```
<h2>
  🚗 Prediction Result
</h2>

{% if prediction == "Important" %}

<div class="prediction important">
  📢 IMPORTANT MESSAGE
</div>

{% else %}

<div class="prediction promotional">
  🛍️ PROMOTIONAL MESSAGE
</div>

{% endif %}

{% if selected_category %}

<div>

  <span class="category">
    {{ selected_category }}
  </span>

</div>

{% endif %}

<div class="message-box">

  <strong>Message:</strong>

  <br><br>

  {{ selected_message }}

</div>

<!-- PROBABILITIES -->

{% if probabilities %}

<div class="probabilities">

  <h3>
    📊 Prediction Probabilities
  </h3>
```

```

    {% for class_name, probability in probabilities.items() %}

    <div class="probability-row">

        <div class="probability-header">

            <span>
                {{ class_name }}
            </span>

            <span>
                {{ probability }}%
            </span>

        </div>

        <div class="bar">

            <div
                class="bar-fill"
                style="width: {{ probability }}%;"
            ></div>

        </div>

    </div>

    {% endfor %}

</div>

{% endif %}

<!-- SAMPLE MESSAGES -->

<div class="card">

    <h2>
         Sample College Messages
    </h2>

    <p class="description">

        Sample messages for testing the trained classifier.

    </p>

```

```

    <div class="message-list">

        {% for item in messages %}

            <div class="message-item">

                <h3>
                    {{ item.category }}
                </h3>

                <p>
                    {{ item.message }}
                </p>

            </div>

        {% endfor %}

    </div>

</div>

<!-- FOOTER -->

<div class="footer">

    Campus Message Detective

    <br>

    Multinomial Naive Bayes + Bag of Words

</div>

</div>

</body>

</html>

```

✓ Flask Code:

```

from flask import Flask, render_template, request
import joblib

app = Flask(__name__)

# Load trained models
vectorizer = joblib.load("vectorizer.pkl")
model = joblib.load("naive_bayes_model.pkl")

```

```

# Sample college messages
messages = [
    {
        "category": "Workshop",
        "message": "A hands-on Artificial Intelligence workshop will be cor
    },
    {
        "category": "Exam",
        "message": "The internal examination timetable has been released. s
    },
    {
        "category": "Placement",
        "message": "The placement cell has announced a campus recruitment c
    },
    {
        "category": "Scholarship",
        "message": "Students eligible for the government scholarship must s
    },
    {
        "category": "College Event",
        "message": "The annual technical festival will be held next week. s
    },
    {
        "category": "Assignment",
        "message": "Students must submit the Machine Learning assignment or
    },
    {
        "category": "Practical",
        "message": "The DBMS practical examination will be conducted in Lat
    },
    {
        "category": "Lecture",
        "message": "Today's Data Science lecture has been shifted to Room :
    },
    {
        "category": "Internship",
        "message": "Students selected for the summer internship must report
    },
    {
        "category": "Coding Competition",
        "message": "Registration is open for the college coding competition
    },
    {
        "category": "Shopping Offer",
        "message": "Exclusive student offer! Get 60 percent off on selected
    },
    {
        "category": "Prize Promotion",
        "message": "Congratulations! You have been selected for a special r
    }
]

```

```
@app.route("/", methods=["GET", "POST"])
```

```
def home():

    prediction = None
    probabilities = None
    selected_message = ""
    selected_category = ""

    if request.method == "POST":

        # Get values from HTML
        selected_index = request.form.get("selected_message")
        custom_message = request.form.get("custom_message", "").strip()

        # Custom message has priority
        if custom_message:

            selected_message = custom_message
            selected_category = "Custom Message"

        # Otherwise use selected sample message
        elif selected_index:

            try:
                index = int(selected_index)

                if 0 <= index < len(messages):

                    selected_message = messages[index]["message"]
                    selected_category = messages[index]["category"]

            except ValueError:
                pass

        # Make prediction
        if selected_message:

            # Convert text into features
            text_vector = vectorizer.transform(
                [selected_message]
            )

            # Predict class
            prediction = model.predict(
                text_vector
            )[0]

            # Get probabilities
            probability_values = model.predict_proba(
                text_vector
            )[0]

            # Create probability dictionary
            probabilities = {}

            for class_name, probability in zip(
```

```
        model.classes_,  
        probability_values  
    ):
```